Практическая работа № 3

Протестировать класс AuthManager, который управляет пользователями, их аутентификацией, а также предоставляет функциональность для подсчета пользователей по странам и перевода средств между ними. В тестах вам нужно продемонстрировать несколько видов тестов: базовые(3 штуки), параметризованные(3 штуки), тестирование исключений(2 штуки), использование фикстур(базы данных) и меток(минимум 2).

Код для тестирования:

```
import sqlite3

class AuthManager:

    def __init__(self, connection):

        self.connection = connection

        self.create_tables()

    def create_tables(self):

        with self.connection:

            self.connection.execute("""

                CREATE TABLE IF NOT EXISTS users (

                    id INTEGER PRIMARY KEY AUTOINCREMENT,

                    username TEXT NOT NULL UNIQUE,

                    password TEXT NOT NULL,

                    country TEXT NOT NULL,

                    balance REAL NOT NULL

                )

            """)

    def register_user(self, username, password, country, balance):

        with self.connection:

            self.connection.execute("""

                INSERT INTO users (username, password, country, balance)

                VALUES ('{}', '{}', '{}', {})

            """.format(username, password, country, balance))    # Уязвимость SQL-инъекции

    def authenticate_user(self, username, password):

        cursor = self.connection.cursor()

        cursor.execute("""

            SELECT * FROM users

            WHERE username = '{}' AND password = '{}'

        """.format(username, password))  # Уязвимость SQL-инъекции
```

```
        return cursor.fetchone()

    def delete_user(self, user_id):
        with self.connection:
            self.connection.execute("""
                DELETE FROM users WHERE id = {}
            """.format(user_id))  # Уязвимость SQL-инъекции
    def get_user_by_id(self, user_id):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT * FROM users WHERE id = {}
        """.format(user_id))  # Уязвимость SQL-инъекции
        return cursor.fetchone()
    def count_users_by_country(self, country):
        cursor = self.connection.cursor()
        cursor.execute("""
            SELECT COUNT(*) FROM users WHERE country = '{}'
        """.format(country))  # Уязвимость SQL-инъекции
        return cursor.fetchone()[0]
    def transfer_balance(self, from_user_id, to_user_id, amount):
        with self.connection:
            # Проверяем, достаточно ли средств
            cursor = self.connection.cursor()
            cursor.execute("SELECT balance FROM users WHERE id =
{}".format(from_user_id))  # Уязвимость SQL-инъекции
            from_balance = cursor.fetchone()[0]
            if from_balance < amount:
                raise ValueError("Insufficient funds")
            # Выполняем перевод
            self.connection.execute("""
                UPDATE users SET balance = balance - {} WHERE id = {}
            """.format(amount, from_user_id))  # Уязвимость SQL-инъекции
            self.connection.execute("""
                UPDATE users SET balance = balance + {} WHERE id = {}
            """.format(amount, to_user_id))  # Уязвимость SQL-инъекции
```

Тест №1 базовый «регистрация пользователя»:

```
@pytest.mark.basic
def test_register_user(auth_manager):
    auth_manager.register_user("alice", "pass123", "CountryA", 500)
    user = auth_manager.authenticate_user("alice", "pass123")
    assert user is not None
    assert user[1] == "alice"
    assert user[3] == "CountryA"
    assert user[4] == 500
```

Результат выполнения теста:



Рисунок 1 – Успешный результат теста

Тест № 2 базовый «аутентификация»:

```
@pytest.mark.basic
def test_authenticate_user_success(filled_auth_manager):
    """Базовый тест: успешная аутентификация."""
    user = filled_auth_manager.authenticate_user("user1", "password123")
    assert user is not None
    assert user[1] == "user1"
```

Результат выполнения теста:

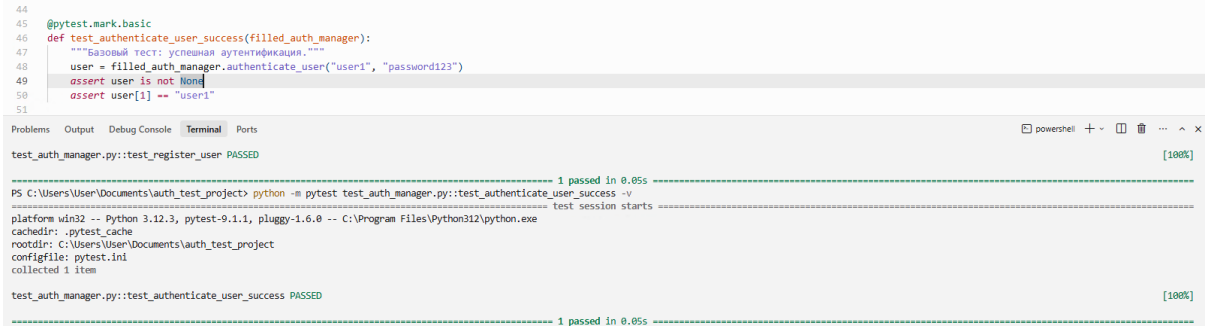


Рисунок 2 – Успешный результат теста

Тест № 3 базовый «перевод средств»:

```
@pytest.mark.basic
def test_transfer_balance_success(filled_auth_manager):
    """Базовый тест: успешный перевод средств."""
    filled_auth_manager.transfer_balance(1, 2, 300)
    user1 = filled_auth_manager.get_user_by_id(1)
    user2 = filled_auth_manager.get_user_by_id(2)
    assert user1[4] == 700
    assert user2[4] == 1300
```

Результат выполнения теста:

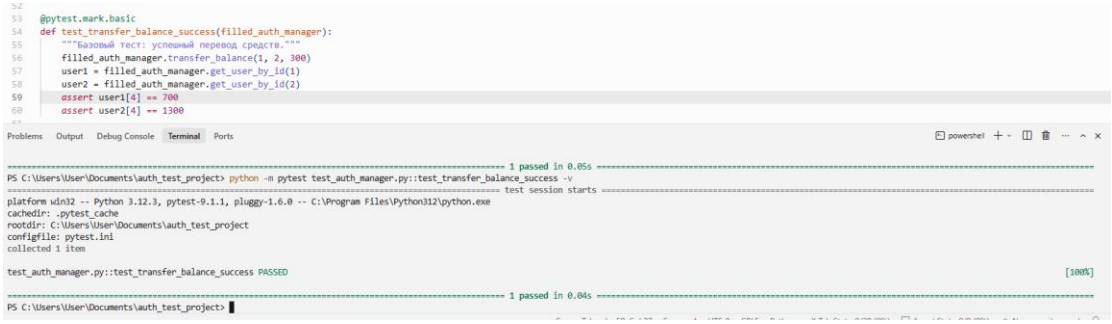


Рисунок 3 – Успешный результат теста

Тест № 4 параметризированный «подсчёт пользователей по странам»:

```
@pytest.mark.parametrize("country, expected_count", [
    ("CountryA", 2),
    ("CountryB", 1),
    ("CountryC", 0), ])
def test_count_users_by_country(filled_auth_manager, country, expected_count):
    """Параметризованный тест: подсчёт пользователей по странам."""
    count = filled_auth_manager.count_users_by_country(country)
    assert count == expected_count
```

Результат выполнения теста:



Рисунок 4 – Успешный результат теста

Тест № 5 параметризированный «аутентификация с разными данными»:

```
@pytest.mark.parametrize("username, password, expected", [
    ("user1", "password123", True),
    ("user1", "wrongpass", False),
    ("nonexistent", "password123", False),
])
def test_authenticate_user_parametrized(filled_auth_manager, username, password, expected):
    """Параметризованный тест: аутентификация с разными данными."""
    user = filled_auth_manager.authenticate_user(username, password)
    assert (user is not None) == expected
```

Результат выполнения теста:



Рисунок 5 – Успешный результат теста

Тест № 6 параметризированный «перевод разных сумм»:

```
@pytest.mark.parametrize("amount, expected_balance", [
    (100, 900),
    (500, 500),
    (1000, 0),
])
def test_transfer_balance_amounts(filled_auth_manager, amount, expected_balance):
    """Параметризованный тест: перевод разных сумм."""
    filled_auth_manager.transfer_balance(1, 2, amount)
    user1 = filled_auth_manager.get_user_by_id(1)
    assert user1[4] == expected_balance
```

Результат выполнения теста:



Рисунок 6 – Успешный результат теста

Тест № 7 исключения «недостаточно средств для перевода»:

```
@pytest.mark.exception
def test_transfer_insufficient_funds(filled_auth_manager):
    """Тест исключения: недостаточно средств для перевода."""
    with pytest.raises(ValueError, match="Insufficient funds"):
        filled_auth_manager.transfer_balance(1, 2, 5000)
```

Результат выполнения теста:

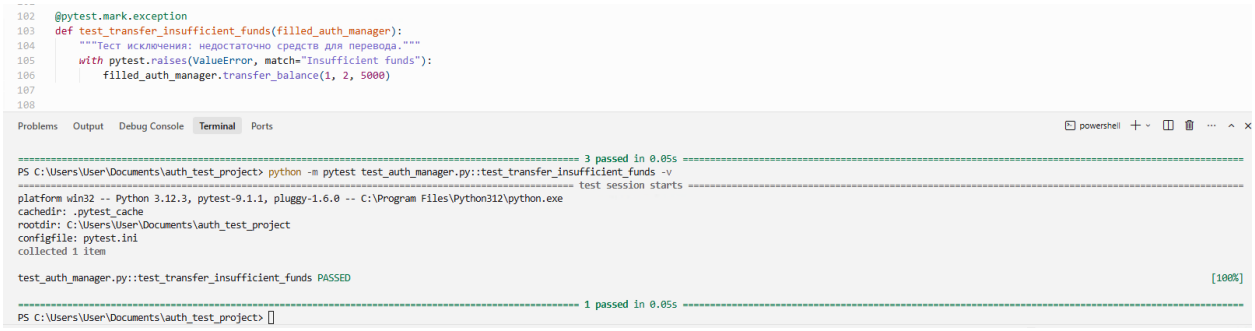


Рисунок 7 – Успешный результат теста

Тест № 8 исключения «аутентификация несуществующего пользователя»:

```
@pytest.mark.exception
def test_authenticate_nonexistent_user(filled_auth_manager):
    """Тест исключения: аутентификация несуществующего пользователя."""
    user = filled_auth_manager.authenticate_user("ghost", "nopass")
    assert user is None
```

Результат выполнения теста:



Рисунок 8 – Успешный результат теста

Тест № 9 SQL-инъекции: аутентификация:

```
@pytest.mark.security
@pytest.mark.sql_injection
def test_sql_injection_login(filled_auth_manager):
    """
    Тест SQL-инъекции: аутентификация.
    """
    malicious_username = "' OR '1'='1"
    malicious_password = "' OR '1'='1"
    user = filled_auth_manager.authenticate_user(
        malicious_username, malicious_password
    )
    assert user is not None, "SQL-инъекция"
```

Результат выполнения теста:



Рисунок 9 – Успешный результат теста

Тест № 10 SQL-инъекции: попытка удалить всех пользователей:

```
@pytest.mark.security
@pytest.mark.sql_injection
```

```
def test_sql_injection_delete(filled_auth_manager):
    """ Тест SQL-инъекции: попытка удалить всех пользователей
    через user_id"""
    malicious_id = "1 OR 1=1"
    filled_auth_manager.delete_user(malicious_id)
    user = filled_auth_manager.get_user_by_id(1)
    assert user is None, "SQL-инъекция должна удалить пользователя"
```

Результат выполнения теста:



Рисунок 10 – Успешный результат теста

Тест № 11, который пропускается:

```
@pytest.mark.skipif(
    not os.path.exists("config.json"),
    reason="Нет файла конфигурации config.json в папке проекта"
)
def test_with_config():
    """Тест, требующий файла config.json в папке проекта."""
    assert True
```

Результат выполнения теста:

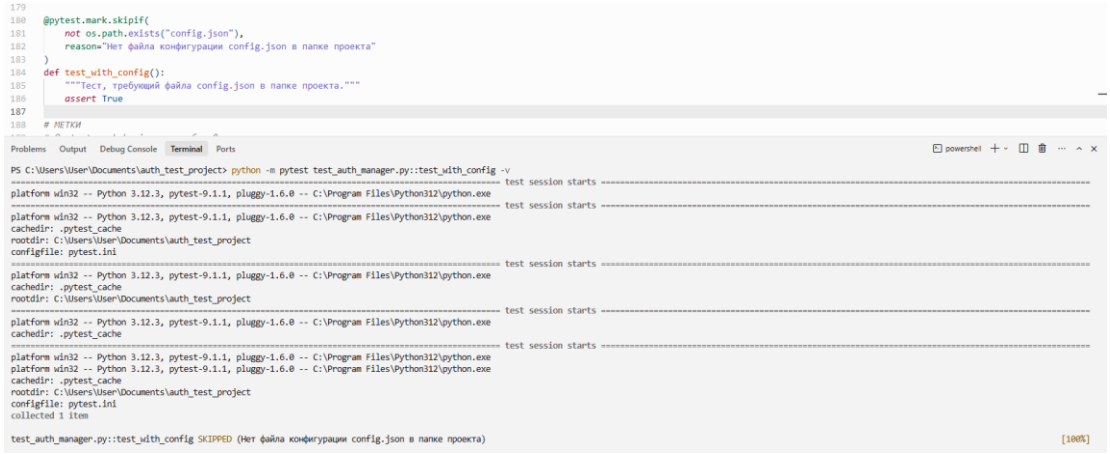


Рисунок 11 – Результат пропущенного теста

Полный листинг кода:

```
import sqlite3
import pytest
import sys
import os

from auth_manager import AuthManager
@pytest.fixture
def db():
    """Фикстура: временная БД SQLite в памяти"""
    connection = sqlite3.connect(":memory:")
    yield connection
    connection.close()

@pytest.fixture
def auth_manager(db):
    """Фикстура: экземпляр AuthManager с тестовой БД"""
    return AuthManager(db)

@pytest.fixture
def filled_auth_manager(auth_manager):
    """Фикстура: AuthManager с предзаполненными пользователями"""
    auth_manager.register_user("user1", "password123", "CountryA", 1000)
    auth_manager.register_user("user2", "password123", "CountryA", 1000)
    auth_manager.register_user("user3", "password123", "CountryB", 1000)
    return auth_manager

@pytest.mark.basic
def test_register_user(auth_manager):

    auth_manager.register_user("alice", "pass123", "CountryA", 500)
    user = auth_manager.authenticate_user("alice", "pass123")
    assert user is not None
    assert user[1] == "alice"
    assert user[3] == "CountryA"
    assert user[4] == 500

@pytest.mark.basic
```

```
def test_authenticate_user_success(filled_auth_manager):
    """Базовый тест: успешная аутентификация"""
    user = filled_auth_manager.authenticate_user("user1", "password123")
    assert user is not None
    assert user[1] == "user1"
@pytest.mark.basic
def test_transfer_balance_success(filled_auth_manager):
    """Базовый тест: успешный перевод средств"""
    filled_auth_manager.transfer_balance(1, 2, 300)
    user1 = filled_auth_manager.get_user_by_id(1)
    user2 = filled_auth_manager.get_user_by_id(2)
    assert user1[4] == 700
    assert user2[4] == 1300


@pytest.mark.parametrize("country, expected_count", [
    ("CountryA", 2),
    ("CountryB", 1),
    ("CountryC", 0),
])

def test_count_users_by_country(filled_auth_manager, country, expected_count):
    """Параметризованный тест: подсчёт пользователей по странам"""
    count = filled_auth_manager.count_users_by_country(country)
    assert count == expected_count



@pytest.mark.parametrize("username, password, expected", [
    ("user1", "password123", True),
    ("user1", "wrongpass", False),
    ("nonexistent", "password123", False),
])
def   test_authenticate_user_parametrized(filled_auth_manager,   username,   password,
expected):
    """Параметризованный тест: аутентификация с разными данными"""
    user = filled_auth_manager.authenticate_user(username, password)
    assert (user is not None) == expected



@pytest.mark.parametrize("amount, expected_balance", [
```

```
    (100, 900),
    (500, 500),
    (1000, 0),
])
def test_transfer_balance_amounts(filled_auth_manager, amount, expected_balance):
    """Параметризованный тест: перевод разных сумм"""
    filled_auth_manager.transfer_balance(1, 2, amount)
    user1 = filled_auth_manager.get_user_by_id(1)
    assert user1[4] == expected_balance

@pytest.mark.exception
def test_transfer_insufficient_funds(filled_auth_manager):
    """Тест исключения: недостаточно средств для перевода"""
    with pytest.raises(ValueError, match="Insufficient funds"):
        filled_auth_manager.transfer_balance(1, 2, 5000)

@pytest.mark.exception
def test_authenticate_nonexistent_user(filled_auth_manager):
    """Тест исключения: аутентификация несуществующего пользователя"""
    user = filled_auth_manager.authenticate_user("ghost", "nopass")
    assert user is None

@pytest.mark.security
@pytest.mark.sql_injection
def test_sql_injection_login(filled_auth_manager):
    """
    Тест SQL-инъекции: аутентификация.
    """
    malicious_username = "' OR '1'='1"
    malicious_password = "' OR '1'='1"

    user = filled_auth_manager.authenticate_user(
        malicious_username, malicious_password
    )
    assert user is not None, "SQL-инъекция"


@pytest.mark.security
```

```
@pytest.mark.sql_injection
def test_sql_injection_delete(filled_auth_manager):
    """
    Тест SQL-инъекции: попытка удалить всех пользователей
    через user_id"""
    malicious_id = "1 OR 1=1"
    filled_auth_manager.delete_user(malicious_id)
    user = filled_auth_manager.get_user_by_id(1)
    assert user is None, "SQL-инъекция должна удалить пользователя"
@pytest.mark.security
@pytest.mark.sql_injection
def test_sql_injection_safe_parameterization(filled_auth_manager):
    # Безопасный запрос с параметрами
    cursor = filled_auth_manager.connection.cursor()
    cursor.execute(
        "SELECT * FROM users WHERE username = ? AND password = ?",
        ("' OR '1'='1", "' OR '1'='1")
    )
    user = cursor.fetchone()

    assert user is None, "Параметризованный запрос защищён от инъекции"
@pytest.mark.skipif(
    not os.path.exists("config.json"),
    reason="Нет файла конфигурации config.json в папке проекта"
)
def test_with_config():
    """Тест, требующий файла config.json в папке проекта"""
    assert True
```